# A*

Starea de explorat se alege după funcția f(s) = g(s) + h(s)

- f(s) – costul total **estimat** al celui mai bun drum de la $s_0$ la o stare finală, trecând prin s
- g(s) – costul celui mai bun drum de la $s_0$ la s
- h(s) – costul **estimat** al celui mai bun drum de la s la o stare finală – estimează h*(s)
- h*(s) – costul **real** al celui mai bun drum de la s la o stare finală
- f*(s) – costul total **real** al celui mai bun drum de la $s_0$ la o stare finală, trecând prin s

Dacă algoritmul este **consistent**, atunci g(s) = g*(s) este costul real al celui mai bun drum de la $s_0$ la s.

Dacă algoritmul este **admisibil**, atunci g(s) = g*(s) pentru stări de pe drumul optim către starea finală.

h(s) este o funcție euristică care estimează h*(s). h(s) este **admisibilă** dacă:

- h(s) ≤ h*(s) (h este o euristică optimistă)
- $h(s_f) = 0$

Dacă h(s) este admisibilă și cost(s', s'') > 0 pentru orice stări s' și s'', atunci **algoritmul A* este admisibil**, adică este garantat că primul drum găsit de la $s_i$ la $s_f$ este și cel optim (din punct de vedere al costului).

Intrare: $s_i$, funcția $h(s)$ Ieșire: soluția, sau INSUCCES

1. OPEN $= \{s_i\}$, CLOSED $= \{\}$, $g(s_i) = 0$, $f(s_i) = h(s_i)$, $\mathbf{p}[s_i] = \perp$
2. **dacă** OPEN $= \{\}$ **atunci întoarce** INSUCCES
3. extrage $s \in$ OPEN, cu $f(s, cale(s, \mathbf{p}))$ *minim*
4. **dacă** $s_j$ este stare finală **atunci întoarce** soluția $cale(s_j, \mathbf{p})$
5. CLOSED $=$ CLOSED $\cup \{s\}$
6. $Succ = calculeaza_succesori(s)$
7. **pentru fiecare** $s_j \in Succ$
8. **dacă** $s_j \in$ CLOSED **și** $g(\{s_j\} \cup cale(s, \mathbf{p})) < g(cale(s_j, \mathbf{p}))$ **atunci** *am găsit o cale mai bună*
9. $\mathbf{p}[s_j] = s$, OPEN $=$ OPEN $\cup \{s_j\}$
10. **dacă** $s_j \notin$ CLOSED **atunci** *starea este nouă*
11. $\mathbf{p}[s_j] = s$
12. CLOSED $=$ CLOSED $\cup \{s_j\}$
13. OPEN $=$ OPEN $\cup \{s_j\}$
14. **mergi la** 2.

Un algoritm $A_1$ este **mai informat** decât $A_2$ dacă $\forall s \in S$, $h_1(s) \geq h_2(s)$ ($h_1$ domină $h_2$). Dacă $A_1$ este mai informat decât $A_2$, ambii algoritmi vor descoperi soluția optimă (dacă sunt admisibili), dar $A_1$ va descoperi soluția optimă mai repede, explorând mai puține stări.

O funcție euristică este **consistentă** dacă pentru $\forall s', s'' \in S$ cu $s'$ părinte al lui $s''$, avem

$$h(s') \leq h(s'') + \text{cost_arc}(s', s'')$$

Putem avea euristici care să fie admisibile, dar să nu fie consistente.

A* explorează un număr mare de stări, mai ales când euristica nu este aproape de costul real. Există strategii de căutare care explorează un număr mai mic de stări:

- Beam Search
- Iterative deepening A* (IDA*)
- Memory-bound A* (MA*), Simplified MA* (SMA*)

**Problemă #1**. Fie 2 algoritmi de tip A*, $A_1$ și $A_2$, ce folosesc $f_1(s) = g(s) + h_1(s)$ și $f_2(s) = g(s) + h_2(s)$ cu $h_1$ și $h_2$ funcții euristice admisibile, $h_1(s) > h_2(s)$, pentru orice s diferit de $s_f$ și $h_1(s_f) = h_2(s_f)$. Selectați afirmația corectă.

a. $A_1$ expandează cel puțin același număr de noduri ca $A_2$
b. $A_1$ și $A_2$ expandează același număr de noduri
c. Nu se poate compara numărul de noduri expandate de $A_1$ cu cele expandate de $A_2$
**d. $A_2$ expandează cel puțin același număr de noduri ca $A_1$**

**Răspuns**. d. $A_2$ expandează cel puțin același număr de noduri ca $A_1$. $h_1$ este o euristică mai informată (oferă o estimare mai bună a costului real rămas). Prin urmare, $A_1$ are mai multe informații pentru a ghida căutarea și va expanda mai puține noduri decât $A_2$.

**Problemă #2**. Care dintre următoarele cazuri poate determina ca algoritmul A* să nu găsească soluția optimă?

a. Utilizarea unei euristici care subestimează costul real până la destinație
**b. Utilizarea unei euristici care supraevaluează costul real până la destinație**
c. Existența mai multor drumuri cu același cost total minim
d. Utilizarea unei cozi de priorități pentru gestionarea nodurilor de expansiune

**Răspuns**. b. Utilizarea unei euristici care supraevaluează costul real până la destinație. Algoritmul A* este garantat să găsească soluția optimă doar când euristica folosită este admisibilă - adică când nu supraevaluează niciodată costul real până la destinație

**Problemă #3**. Care dintre următoarele afirmații despre euristicile admisibile și consistente în algoritmul A* este adevărată?

a. Orice euristică admisibilă este și consistentă
**b. Orice euristică consistentă este și admisibilă**

c. A* cu o euristică admisibilă dar inconsistentă garantează soluția optimă doar dacă se folosește redeschiderea nodurilor închise
d. A* cu o euristică consistentă necesită mai multă memorie decât A* cu o euristică doar admisibilă

**Răspuns**. b. Orice euristică consistentă este și admisibilă. Consistența este o condiție mai puternică decât admisibilitatea. Dacă o euristică este consistentă, atunci este automat și admisibilă.

**Problemă #4**. Un robot trebuie să navigheze într-un oraș reprezentat ca o grilă 2D, unde fiecare celulă poate fi liberă sau poate conține un obstacol. Robotul se poate deplasa în cele patru direcții cardinale cu un cost de 1 per mișcare. Care dintre următoarele euristici este cea mai potrivită pentru algoritmul A* în acest scenariu?

a. Distanța Euclidiană până la destinație
**b. Distanța Manhattan până la destinație**
c. Numărul de obstacole între poziția curentă și destinație
d. Diferența de elevație între poziția curentă și destinație

**Răspuns**. b. Distanța Manhattan până la destinație. Este admisibilă – Nu supraestimează niciodată costul real (deoarece robotul nu se poate deplasa pe diagonală). Este consistentă – Întotdeauna respectă $h(s') \leq 1 + h(s'')$ pentru orice mișcare cu cost 1.

**Problemă #5**. Dacă un algoritm A* folosește o euristică consistentă, atunci:

a. Nu se poate preciza optimalitatea soluției
b. Găsește drumul optim doar până la starea finală
**c. Găsește drumul optim până la orice stare pe care o descoperă**
d. Găsește drum până la starea finală, dar nu este cel optim

**Răspuns**. c. Găsește drumul optim până la orice stare pe care o descoperă. A* găsește cel mai scurt drum nu doar până la destinație, ci și până la orice alt nod explorat pe parcurs. Când A* expandează un nod, deja cunoaște cel mai bun drum până la el.

**Problemă #6**. Dacă euristica pentru un algoritm A* depășește costul optim până la soluție cu cel mult 1, atunci:

a. Costul soluției descoperite este de două ori costul soluției optime
b. Algoritmul nu găsește soluția
**c. Costul soluției descoperite depășește costul soluției optime cu cel mult 1**
d. Este posibil ca algoritmul să descopere o soluție invalidă

**Răspuns**. c. Costul soluției descoperite depășește costul soluției optime cu cel mult 1. Limita superioară a erorii euristicii (+1) se reflectă direct în costul soluției găsite.

## Beam Search

Combină BFS cu o abordare greedy și cu utilizarea unei euristici. Păstrează în frontieră doar un număr B de stări, selectate conform cu o euristică h(s). Alegându-se o stare din frontieră, se adaugă toți succesorii săi la frontieră, dar apoi din frontieră se selectează doar cele mai bune B stări.

Algoritmul nu este optimal sau complet. Poate deveni complet dacă este combinat cu DFS în:

- Beam Stack Search
- Limited Discrepancy Search

**Beam-Search**(*start*, *B*, *h*, *limita*)

1. *beam* = {*start*}
2. *vizitat* = {*start*}
3. **cât timp** *beam* ≠ ∅ și |*vizitat*| < *limita*
4. *Succ* = ∅
5. **pentru fiecare** *s* ∈ *beam*
6. *Succ* = *Succ* ∪ *calculeaza_succesori*(*s*) *vizitat*
7. **dacă** ∃*s* ∈ *Succ*, *s* stare finală
8. **atunci întoarce** *SUCCES*
9. *selectat* = cele mai bune *B* stări din *Succ*, sortate după *h*(*s*)
10. *vizitat* = *vizitat* ∪ *selectat*
11. *beam* = *selectat*

## IDA*

Combină ID-DFS cu A*, limitând adâncimea explorării la o limită care folosește funcția f din A*. Explorarea se face doar până la o anumită adâncime fixată, inițial f($s_0$). Când această limită este atinsă, se extinde până la f(s*), cu s* cea mai bună stare găsită până acum.

**Algoritm IDA***

1. *limita* = *h*($s_0$), *s* = $s_0$, **p**[$s_0$] = ⊥
2. **repetă**
3. *r* = *DF*(*s*, 0, *limita*)
4. **dacă** *r* = ⟨$s_0$, ..., $s_f$⟩ **atunci întoarce** *r* — *succes – a fost întoarsă o soluție*
5. **dacă** *r* = INSUCCES **atunci întoarce** *r*
6. *limita* = *r* — *altfel, a fost întoarsă o nouă limită*

1. *DF*(*s*, *g*, *limita*)
2. **dacă** *g* + *h*(*s*) > *limita* **atunci întoarce** *g* + *h*(*s*) — *este prevăzut că soluția va depăși limita*
3. **dacă** *s* este stare finală **atunci întoarce** *cale*(*s*, **p**)
4. *min* = INSUCCES
5. **pentru fiecare** $s_j$ ∈ *succesori*(*s*)
6. **dacă** $s_j$ ∉ *cale*(*s*, **p**)
7. **p**[$s_j$] = *s*
8. *r* = *DF*($s_j$, *g* + *cost_arc*(*s*, $s_j$), *limita*) — *caută în adâncime*
9. **dacă** *r* = ⟨$s_0$, ..., $s_f$⟩ **atunci întoarce** *r* — *întoarce cale*
10. **dacă** *r* ≠ INSUCCES **și** *min* = INSUCCES **sau** *r* < *min* **atunci** *min* = *r*
11. **întoarce** *min*

Explorează mai în direcția soluției decât DFS, pentru că zona explorată este ținută la o anumită distanță estimată față de soluție. Consumă mai puțină memorie decât A*, pentru că nu menține structurile OPEN și CLOSED.

Exemple de funcții euristice

În probleme de drum de cost optim într-un grid:

- Distanța Manhattan
- Distanța Euclidiană

În problema comis-voiajorului:

- $h_1(s) = cost_arc(s_i, s)$
- $h_2(s)$ = costul arborelui de acoperire de cost minim al orașelor neparcurse până în starea s

În jocul 8-puzzle:

- $h_1$ – numărul pătratelor care nu sunt pe poziția lor finală
- $h_2$ / $h_3$ – suma distanțelor Euclidiene / Manhattan până la poziția finală pentru fiecare pătrat care nu este la locul său
- $h_4$ – numărul pătratelor care nu sunt pe rândul corespunzător + numărul pătratelor care nu sunt pe coloana corespunzătoare

## Căutare locală

Este o strategie de căutare bazată pe mutarea dintr-o stare curentă într-o stare vecină. Este utilă când:

- spațiul de căutare este mare și / sau nu avem suficient spațiu pentru a păstra în memorie un număr mare de stări
- nu ne interesează optimalitatea drumului până la soluție
- ne interesează starea finală în care ajungem

Este util în probleme de tip CSP.

**Problemă #7**. Avantajul unei căutări locale este că:

a. Reținem drumul pe care am explorat în așa fel încât să ne putem întoarce
**b. Avem complexitate spațială redusă**
c. Nu este niciodată nevoie, pentru a găsi soluția optimă, să mergem într-o stare care nu este vecină

d. Nu avem nevoie de informare asupra stărilor problemei

**Răspuns**. b. Avem complexitate spațială redusă.

## Hill Climbing

Alege întotdeauna o stare vecină pentru care o funcție de evaluare este mai bună.

```
Hill Climbing(s_0, eval, limita)
1. s ← s_0
2. cât timp iter < limita
3.     iter ← iter + 1
4.     Succ ← calculeaza_succesori(s)
5.     dacă ∃s_j ∈ Succ, s_j stare finală atunci întoarce s_j
6.     găsește s' ∈ Succ cu eval(s') ≥ eval(s)
7.     s ← s'
8. întoarce EȘEC
```

Hill Climbing găsește soluția atunci când funcția eval este convexă pe spațiul stărilor. Dacă există mai multe puncte de maxim local, este posibil ca Hill Climbing să găsească un maxim local. Dacă există platouri, este posibil ca Hill Climbing să se blocheze.

Pentru a rezolva o parte din problemele care pot apărea, putem folosi **Random Restart Hill Climbing**.

```
Random Restart Hill Climbing(N, eval, limita)
1. pentru i = 1, N
2.     alege o stare s_0
3.     rezultat ←Hill Climbing(s_0, eval, limita)
4.     dacă rezultat ≠ EȘEC atunci întoarce rezultat
5. întoarce EȘEC
```

Dacă se observă că o problemă Hill Climbing are probabilitate p de reușită, putem alege N = 1 / p.

**Problemă #8**. Care dintre următoarele reprezintă cea mai mare problemă a algoritmului Hill Climbing?

a. Consumul mare de memorie
**b. Blocarea în maxime locale**
c. Timpul de execuție exponențial
d. Necesitatea unui set de date etichetat

**Răspuns**. b. Blocarea în maxime locale. Algoritmul se "blochează" aici și nu poate scăpa fără mecanisme suplimentare (e.g., *Random Restart*, *Simulated Annealing*).

**Problemă #9**. Care este principalul avantaj al algoritmului Random Restart Hill Climbing față de varianta standard a Hill Climbing?

a. Reduce semnificativ consumul de memorie
b. **Crește probabilitatea de a găsi optimul global prin evitarea blocării în maxime locale**
c. Garantează convergența la soluția optimă pentru toate tipurile de probleme
d. Elimină complet nevoia de a defini funcții de vecinătate

**Răspuns**. b. Crește probabilitatea de a găsi optimul global prin evitarea blocării în maxime locale.

## Simulated Annealing

Considerăm că stările au o anumită energie și căutăm starea de energie minimă globală. Mergem într-o stare vecină mai bună dacă am găsit una, dar putem merge, cu o anumită probabilitate, și într-o stare mai proastă. Probabilitatea de a alege o stare mai proastă scade pe măsură ce algoritmul progresează. Dacă algoritmul este bine proiectat, poate găsi minimul global și atunci când există un număr mare de minime locale.

Simulated Annealing($T_{init}$, $T_{final}$, *cooling*, *eval*, *alege_succesor*)

1. $s \leftarrow s_0$ ; $T \leftarrow T_{init}$
2. **cât timp** $T > T_{final}$
3. $s' \leftarrow alege_succesor(s)$
4. **dacă** $eval(s') < eval(s)$
5. **atunci** $s \leftarrow s'$
6. **altfel** $s \leftarrow s'$ cu probabilitate $e^{-\frac{eval(s') - eval(s)}{T}}$
7. $T \leftarrow cooling \cdot T$

Graful stărilor trebuie să aibă un **diametru redus** – chiar dacă numărul de stări este foarte mare, trebuie să existe un drum relativ scurt între orice stare și starea finală.

Poate fi util ca generarea stărilor succesor să producă stări de **energie apropiată** cu starea curentă, pentru a evita mutarea în stări foarte proaste.

Algoritmul este vulnerabil la seturi mari de stări cu energie foarte mică față de vecinii lor, dar nu global minimă. Se pot folosi **reporniri** – reveniri la stări de energie mică obținute în trecut, dacă au trecut un număr de pași cu energie semnificativ mai mare de minima găsită.

**Problemă #10**. Care dintre următoarele reprezintă un dezavantaj major în aplicarea unui algoritm de tip Simulated Annealing?

a. Generarea de stări cu energie apropiată ca vecini a stării curente
b. Un număr mare de minime sau maxime locale
c. Existența unor stări cu foarte mulți vecini

**d. Un număr mare de tranziții necesare pentru a ajunge între perechi de stări ale problemei**

**Răspuns**. d. Un număr mare de tranziții necesare pentru a ajunge între perechi de stări ale problemei. Dacă spațiul de căutare are stări izolate sau drumuri lungi între stări promițătoare, algoritmul poate necesita foarte multe iterații pentru a explora eficient spațiul.

**Problemă #11**. Care dintre următoarele opțiuni facilitează găsirea unei soluții mai bune într-o căutare locală folosind algoritmul Simulated Annealing? (1) Scăderea accelerată a parametrului temperatură. (2) Considerarea unor stări succesive cu valoare de energie apropiată cu cea a stării curente. (3) Utilizarea repornirilor prin revenirea la stări cu energie foarte mare găsite anterior. (4) Utilizarea unei temperaturi inițiale mai mici.

a. (1) și (4)
**b. (2) și (3)**
c. (2)
d. (4)

**Răspuns**. b. (2) și (3). (2) – Aceasta permite o explorare mai stabilă a spațiului de căutare, evitând salturi prea mari care ar putea sări peste soluții bune. (3) – Revenirea la stări promițătoare (chiar dacă nu sunt cele mai bune la un moment dat) poate deschide noi căi de explorare.

**Problemă #12**. Care este rolul principal al parametrului de temperatură în algoritmul Simulated Annealing?

a. Determină viteza de execuție a algoritmului
**b. Controlează probabilitatea de a accepta soluții mai slabe temporar**
c. Stabilește numărul maxim de iterații
d. Definește metrica de distanță între stări

**Răspuns**. b. Controlează probabilitatea de a accepta soluții mai slabe temporar. La temperaturi mari, algoritmul acceptă cu probabilitate ridicată și soluții mai slabe decât cea curentă, permițând o explorare mai amplă a spațiului de căutare. Pe măsură ce temperatura scade, probabilitatea de a accepta soluții mai slabe scade, iar algoritmul converge spre o regiune optimă (similar cu procesul de răcire din metalurgie).

**Problemă #13**. Care dintre următoarele reprezintă un dezavantaj al algoritmului Simulated Annealing în comparație cu Random Restart Hill Climbing?

**a. Necesită mai mult timp de execuție pentru a atinge convergența datorită evaluării graduale a soluțiilor potențial suboptimale**
b. Nu poate fi aplicat pentru probleme cu spațiu de stare discret
c. Are o complexitate spațială semnificativ mai mare
d. Nu poate fi paralelizat eficient pentru rulare pe sisteme distribuite

**Răspuns**. a. Necesită mai mult timp de execuție pentru a atinge convergența datorită evaluării graduale a soluțiilor potențial suboptimale. Simulated Annealing folosește o abordare probabilistică, acceptând temporar soluții mai slabe pentru a evita optimalele locale, cu o probabilitate care scade pe măsură ce "temperatura" scade. Random Restart Hill Climbing rulează Hill Climbing de mai multe ori pornind din puncte aleatoare, fără a accepta soluții mai slabe.

## Căutarea on-line

În căutarea **off-line**, agentul are acces la toate stările problemei și poate alege să calculeze succesorii oricărei stări în orice moment. În căutarea **on-line**, agentul poate alege ca stare următoare doar o stare vecină cu starea curentă – agentul realizează căutarea în timp ce acționează în mediu.

Exemple: într-o problemă de căutare on-line, Hill Climbing nu poate fi folosit în conjuncție cu Random Restart; similar, Simulated Annealing funcționează doar dacă alege_succesor alege o stare vecină în mediul fizic al căutării; BFS nu se poate aplica; la DFS, revenirea aduce un cost; la A* nu putem alege orice nod pentru a fi explorat la următoarea iterație.

**Problemă #14**. Care dintre următoarele nu reprezintă o diferență corectă între algoritmii de căutare locală și algoritmii de căutare globală?

a. Algoritmii de căutare locală utilizează mai puțină memorie decât algoritmii de căutare globală
b. Algoritmii de căutare globală garantează întotdeauna găsirea soluției optime, în timp de cei de căutare locală nu
c. Algoritmii de căutare locală operează de obicei cu o singură stare curentă, iar cei de căutare globală mențin o frontieră de stări
**d. Algoritmii de căutare locală sunt adesea utilizați pentru probleme de optimizare, în timp ce cei de căutare globală sunt mai potriviți pentru probleme de satisfacere a constrângerilor**

**Răspuns**. d. Algoritmii de căutare locală sunt adesea utilizați pentru probleme de optimizare, în timp ce cei de căutare globală sunt mai potriviți pentru probleme de satisfacere a constrângerilor. Nu există o asociere exclusivă între tipul de algoritm și categoria de probleme.

**Problemă #15**. Într-o căutare online, o dificultate importantă vine din faptul că:

**a. Pentru a schimba starea, este nevoie să parcurgem calea până în noua stare**
b. Frontiera poate conține un număr nelimitat de stări
c. Complexitatea spațială este mai mare
d. Nu putem reține stările în care trebuie să ne întoarcem

**Răspuns**. a. Pentru a schimba starea, este nevoie să parcurgem calea până în noua stare. În căutarea offline, putem "sări" direct între stări în memorie. În căutarea online, tranzițiile sunt fizice sau costisitoare în timp.

## Asynchronous Dynamic Programming – ADP

În programarea dinamică se lucrează prin obținerea soluției la diverse sub-probleme, care pot să nu fie disjuncte. Soluția globală este optimă dacă orice componentă a soluției este soluție optimă pentru o subproblemă. Avem nevoie de acces la întreg spațiul de stări pentru a folosi DP. Pentru spații de stări foarte mari, este costisitor să parcurgem chiar și o singură dată întreg spațiul.

Dacă am cunoaște h* pentru fiecare nod, am putea obține drumul optim, prin căutare locală, astfel:

1. **cât timp** nu am ajuns la starea finală
2. **pentru fiecare** fiecare $s_j \in$ *calculeaza_succesori*$(s)$
3. $f^*(s_j) = c(s, s_j) + h^*(s_j)$ cu $c(a, b)$ costul muchiei $(a, b)$
4. $s \leftarrow argmin_{s_j} f^*(s_j)$

Putem actualiza asincron estimarea h[s] pentru fiecare stare astfel:

1. **alege** o stare $s$ arbitrară
2. for fiecare $s_j \in$ *calculeaza_succesori*$(s)$
3. $f(s_j) = c(s, s_j) + h[s_j]$
4. $h[s] \leftarrow min_{s_j} f(s_j)$

Pentru costuri pozitive, obținem convergență, dar pentru spații de stări mari, este nepractic.

## LRTA*

Se folosește o tabelă cu valorile estimate h[s] pentru fiecare stare. Actualizăm valorile pe măsură ce explorăm. Trebuie să considerăm costul pentru orice revenire. Revenim doar dacă costul estimat pe calea curentă depășește costul implicat de revenire.

LRTA*($s$) – întoarce acțiunea de realizat din $s$

global: $H$, $Rezultat$, $s_{prev}$, $a_{prev}$ inițial, $s_{prev} \leftarrow \perp$

1. **dacă** $s$ este stare finală **atunci întoarce** SUCCES
2. **dacă** $s$ este stare nouă **atunci** $H[s] \leftarrow h(s)$ cu $h$ funcția euristică
3. **dacă** $s_{prev} \neq \perp$ **atunci**
4. $Rezultat[s_{prev}, a_{prev}] \leftarrow s$
5. $H[s_{prev}] \leftarrow min_{b \in actiuni(s_{prev})} \begin{cases} c(s_{prev}, s') + H[s'] & \text{dacă există } s' = Rezultat[s_{prev}, b] \\ h(s_{prev}) & \text{, altfel} \end{cases}$
6. $s_{prev} \leftarrow s$
7. **întoarce** $b \in actiuni(s)$,

care minimizează $\begin{cases} c(s, s'') + H[s''] & \text{dacă există } s'' = Rezultat[s, b] \\ h(s) & \text{, altfel} \end{cases}$

Dacă toate costurile sunt pozitive, există o cale de la orice stare la starea finală și funcția h este admisibilă, atunci LRTA* este complet. LRTA* se apropie de soluția optimă la parcurgeri succesive.

## Constraint Satisfaction Problem – CSP

$\{X_1 \dots X_N\} = V$ – *Variable*

$\{D_1 \dots D_N\}$ – *Domenii*

$\{R_1 \dots R_k\} = R$ – *Restricții*, $R_j = \langle t_j, r_j \rangle$, $t_j \subseteq V$, $r_j : \times_{X_i \in t_j} D_i \rightarrow \{True, False\}$

$\{\langle X_1, x_1 \rangle \dots \langle X_N, x_N \rangle\}$ – Atribuire la *valori*, cu $x_i \in D_i \cup \{\perp\}$ ($\perp \equiv$ neatribuit la o valoare)

o atribuire este *soluție* dacă

- $x_i \in D_i, i = \overline{1, N}$, i.e. toate variabilele au atribuită o valoare (*atribuire completă*)
- $\forall R_j \in R, r_j(vals) = True$, cu *vals* valorile variabilelor din $t_j$ (*soluție validă*)

CSP **totală** – soluția este completă și nu încalcă nicio restricție

CSP **parțială** – soluția este completă dar încalcă unele restricții

$R_j = \langle t_j, r_j, c_j \rangle$, cu $c_j$ costul restricției $R_j$

costul soluției $c = \sum_{R_j \in R, r_j(vals(t_j)) = False} c_j$

CSP **binară** – toate restricțiile sunt binare

$\forall R_j \in R, t_j \in V \times V$

**Problemă #16**. Folosim algoritmi pentru CSP Parțială atunci când, spre deosebire de CSP Totală:

a. Este suficient să atribuim valori doar pentru o parte din variabile
b. Este suficient să folosim doar o parte din domeniul fiecărei variabile pentru atribuirea de valori
**c. Este suficient să satisfacem doar o parte din restricții**
d. Avem exact aceleași cerințe pentru soluția de la CSP completă și de la CSP parțială

**Răspuns**. c. **Este suficient să satisfacem doar o parte din restricții.** CSP **parțială** – soluția este completă dar încalcă unele restricții

## Backtracking

CSP este o problemă NP, trebuie rezolvat prin backtracking. Ce putem face este să încercăm să reducem spațiul de căutare și factorul de ramificare.

**CSP-BKT**
Intrări: *Vars* – variabile rămase de atribuit, inițial $X$;
*Sol* – soluția (atribuirea) parțială, inițial $\emptyset$
Ieșire: O atribuire completă sau EȘEC

1. **dacă** $Vars = \emptyset$ **atunci întoarce** *Sol*
2. $X_i \leftarrow$ *alege_variabila*(*Vars*)
3. **pentru fiecare** $x \in D_i$
4. **dacă** *consistent*$(Sol \cup \langle X_i, x \rangle, R)$ **atunci**
5. $res \leftarrow$ *CSP-BKT*$(Vars \setminus \{X_i\}, Sol \cup \langle X_i, x \rangle)$
6. **dacă** $res \neq$ EȘEC **atunci întoarce** *res*
7. **întoarce** EȘEC

## Algoritmul AC-3 (Arc Consistency 3)

Trebuie ca toate restricțiile să fie unare sau binare. Putem forma un graf de restricții în care nodurile corespund variabilelor și arcele corespund restricțiilor binare. Considerăm pentru fiecare restricție două arce orientate.

un arc $(X_i, X_j)$ corespunzător unei restricții $R_{ij}$ este *arc-consistent* dacă

$$\forall x_i \in D_i, \exists x_j \in D_j, r_{ij}(x_i, x_j) = True$$

Dacă toate arcele din graf sunt arc-consistente, graful este arc-consistent și nu putem reduce mai mult (prin acest mecanism) spațiul de căutare.

**AC-3**

1. reduce toate domeniile conform cu restricțiile unare
2. $Q \leftarrow \{(X_i, X_j) \mid R_{ij} \in R \text{ sau } R_{ji} \in R\}$
3. **cât timp** $Q \neq \emptyset$
4. $(X_i, X_j) \leftarrow Q.pop()$
5. **dacă** *check*$(X_i, X_j)$ **atunci** *domeniul lui* $X_i$ *s-a modificat*
6. **dacă** $D_i = \emptyset$ **atunci întoarce** EȘEC
7. $Q = Q \cup \{(X_k, X_i) \mid R_{ki} \in R, k \neq j\}$
8. **întoarce** SUCCES

*check*$(X_i, X_j)$

1. **pentru** $x_i \in D_i$
2. **dacă** $\nexists x_j \in D_j \; . \; r_{ij}(x_i, x_j) = True$
3. **atunci** $D_i \leftarrow D_i \setminus \{x_i\}$
4. **întoarce** *True* **dacă** $D_i$ s-a modificat, **altfel** *False*

Complexitatea temporală pentru AC-3 este $O(e \cdot d^3)$, unde

- $e = |R|$ – numărul de restricții (muchii)
- $d = max_{i=\overline{1,N}}|D_i|$ – cardinalitatea maximă a domeniilor

Complexitatea spațială este O(e).

Algoritmi mai noi, dar mai complicați, pot fi mai buni. De exemplu, AC-4 are complexitate $O(e \cdot d^2)$. Dacă AC-3 întoarce EȘEC atunci înseamnă că problema nu are soluție.

Dacă după realizarea AC-3, toate domeniile au cardinalitate 1 (spunem că lățimea grafului de restricții este 1), atunci nu mai este necesar backtracking. Dacă lățimea grafului de restricții este mai mare de 1, atunci nu este garantat că problema are soluție și trebuie făcut backtracking pe variabile cu domenii de variabilitate > 1.

**Problemă #17**. Este adevărat despre algoritmul AC-3 că:

e. Reduce domeniile tuturor variabilelor la o singură valoare
f. Nu prelucrează un nod de mai multe ori
**g. Reduce factorul de ramificare în căutarea soluției prin backtracking**
h. Complexitatea temporală a algoritmului nu depinde de numărul de noduri

**Răspuns**. c. Reduce factorul de ramificare în căutarea soluției prin backtracking. AC-3 este folosit ca preprocesare pentru backtracking. Prin reducerea dimensiunii domeniilor, el scade numărul de ramificări necesare în timpul căutării, accelerând rezolvarea.

**Problemă #18**. Folosirea AC-3 ne avantajează pentru că:

a. Complexitatea sa temporală nu depinde de numărul de noduri
b. Un graf de restricții care este arc-consistent nu necesită niciodată backtracking
**c. În anumite cazuri ne poate arăta că problema nu are soluție**
d. După aplicarea AC-3 toate nodurile din arborele de explorare devin binare

**Răspuns**. c. În anumite cazuri ne poate arăta că problema nu are soluție. Dacă AC-3 întoarce EȘEC atunci înseamnă că problema nu are soluție.

**Problemă #19**. Care dintre următoarele reprezintă o limitare semnificativă a algoritmului AC-3 în rezolvarea problemelor CSP complexe?

a. Necesită ca toate variabilele să aibă domenii de aceeași dimensiune
b. Are nevoie de o implementare recursivă care poate duce la epuizarea stivei pentru probleme mari
c. Funcționează doar pe grafuri de constrângeri care sunt arbori sau care nu conțin cicluri
**d. Nu poate detecta inconsistențele care necesită examinarea a trei sau mai multe variabile simultan**

**Răspuns**. d. Nu poate detecta inconsistențele care necesită examinarea a trei sau mai multe variabile simultan. AC-3 lucrează doar cu constrângeri binare (între două variabile) și nu poate detecta inconsistențe care implică trei sau mai multe variabile simultan (de exemplu, constrângeri ternare).

**Problemă #20**. Utilizarea tehnicilor de verificare a arc-consistenței în problema satisfacerii restricțiilor este de a:

a. Calcula timpul în care se va termina căutarea soluției
**b. Reduce spațiul de căutare pentru backtracking**
c. Verifica dacă atribuirea curentă este soluție a problemei
d. Verifica dacă există o soluție mai bună pe o altă ramură a arborelui de căutare

**Răspuns**. b. Reduce spațiul de căutare pentru backtracking. Tehnicile de verificare a arc-consistenței (cum ar fi algoritmul AC-3) sunt utilizate pentru a elimina valorile inconsistente din domeniile variabilelor în problemele de satisfacere a constrângerilor

## Îmbunătățiri backtracking

- Maintaining arc-consistency – MAC

Putem reduce spațiul de căutare prin detectarea timpurie a inconsistențelor. Combinăm backtracking-ul cu arc-consistența, menținând arc-consistența pe măsură ce avansăm în recursivitate. Verificăm arc-consistența după ce atribuim o valoare în soluția parțială.

**BKT-MAC**

1. **dacă** $Vars = \emptyset$ **atunci întoarce** $Sol$
2. $X_i \leftarrow alege_variabila(Vars)$
3. **pentru fiecare** $x \in D_i$
4. $D_{copy} \leftarrow D$ — *copiem toate domeniile*
5. $r \leftarrow AC\text{-}3'(arcs_out(X_i))$ — *arcs_out* întoarce toate arcele care pornesc din $X_i$; AC-3' este AC-3 începând de la pasul 3., cu $Q$ pornind de la valoarea din argument
6. **dacă** $r$ **și** $consistent(Sol \cup \langle X_i, x \rangle, R)$ **atunci**
7. $res \leftarrow BKT\text{-}MAC(Vars \setminus \{X_i\}, Sol \cup \langle X_i, x \rangle)$
8. **dacă** $res \neq$ EȘEC **atunci întoarce** $res$
9. $D \leftarrow D_{copy}$
10. **întoarce** EȘEC — *refacem domeniile*

- Ordonarea variabilelor

Putem ordona variabilele (funcția alege_variabila):

- Aleator
- Alegând variabila cea mai restricționată (cu domeniul cel mai redus) – Minimum remaining value – MRV; este util atunci când ne dorim o singură soluție și vrem să forțăm, dacă e cazul, un eșec mai rapid
- Alegând variabila cea mai puțin restricționată; util atunci când ne dorim aflarea tuturor soluțiilor și ne dorim un arbore BKT mai lat și mai puțin adânc
- Alegând variabila implicată în cele mai multe restricții cu variabile încă neatribuite – Degree heuristic

- Ordonarea valorilor

Putem parcurge valorile din domeniu într-o anumită ordine:

- Selectăm întâi valoarea care elimină cele mai puține valori din variabilele vecine la menținerea arc-consistenței; util atunci când ne dorim o singură soluție și vrem să ajungem la o soluție mai repede
- Selectăm întâi valoarea care elimină cele mai multe valori din variabilele vecine la menținerea arc-consistenței; util atunci când dorim toate soluțiile și preferăm un graf mai puțin adânc

# PCSP

Pentru o problemă de tip CSP cu multe restricții, este posibil ca nu toate restricțiile să file la fel de importante.

- asociem un cost cu fiecare restricție: $R_j = \langle t_j, r_j, c_j \rangle$,cu $c_j > 0$
- stabilim o limită suficientă $S$ – considerăm o soluție *Sol* ca validă dacă $\sum_{j, r_j(Sol)=False} c_j \leq S$
- optimizare: dacă în parcurgere costul total al restricțiilor încălcate de atribuirea parțială curentă depășește S, sau depășește costul restricțiilor pentru cea mai bună soluție găsită până acum, nu are sens să mai continuăm pe aceast cale

**PCSP** – găsește prima soluție acceptabilă
Intrări: *Vars* – variabile rămase de atribuit, inițial $X$;
*Sol* – soluția (atribuirea) parțială, inițial $\emptyset$
*cost* – costul soluției parțiale *Sol*, cu $S$ – limita suficientă
Ieșire: O atribuire completă de cost $\leq S$ sau EȘEC

1. **dacă** $Vars = \emptyset$ **atunci întoarce** *Sol*
2. $X_i \leftarrow$ *alege_variabila*(*Vars*)
3. **pentru fiecare** $x \in D_i$
4. $Sol' \leftarrow Sol \cup \langle X_i, x \rangle$
5. $c = cost + \sum_{X_i \in t_j, r_j(Sol')=False} c_j$
6. **dacă** $c \leq S$
7. $res \leftarrow PCSP(Vars \setminus \{X_i\}, Sol', c)$
8. **dacă** $res \neq$ EȘEC **atunci întoarce** *res*
9. **întoarce** EȘEC

**PCSP – găsește cea mai bună soluție acceptabilă**
Intrări: *Vars* – variabile rămase de atribuit, inițial $X$;
*Sol* – soluția (atribuirea) parțială, inițial $\emptyset$
*cost* – costul soluției parțiale *Sol*, cu $S$ – limita suficientă
Global: *sols* – toate soluțiile complete acceptabile găsite până acum
*best_cost* – costul minim al unei soluții din *sols*
Ieșire: O mulțime de atribuiri complete de cost $\leq S$, sau EȘEC

1. **dacă** $Vars = \emptyset$ **atunci**
2. $sols \leftarrow sols \cup \{Sol\}$
3. **dacă** $cost < best_cost$ **atunci** $best_cost \leftarrow cost$
4. $X_i \leftarrow$ *alege_variabila*(*Vars*)
5. **pentru fiecare** $x \in D_i$
6. $Sol' \leftarrow Sol \cup \langle X_i, x \rangle$
7. $c = cost + \sum_{X_i \in t_j, r_j(Sol')=False} c_j$
8. **dacă** $c \leq S$ **și** $c < best_cost$
9. $PCSP(Vars \setminus \{X_i\}, Sol', c)$ *va actualiza sols dacă este cazul*

## Căutare în jocuri

Vedem un joc ca o problemă de căutare – are un spațiu de stări, o stare inițială și stări finale în care jocul se termină și în care jucătorii primesc o recompensă.

- Jocuri
  - Cooperative – Pandemic
  - Non-cooperative – Poker
  - Cu sumă 0 – X și 0, Șah, Go
- Jocuri cu informație
  - Perfectă – Șah, X și 0, Go
  - Imperfectă – Poker, Bridge
  - Cu elemente de șansă – Table, Poker
- Jocuri
  - Secvențiale – X și 0, Șah, Poker
  - Cu mișcări simultane – Rock-paper-scissors

## Jocuri cu 2 adversari

Ne referim mai ales la jocuri

- Cu sumă 0 – un jucător pierde și altul câștigă
- Cu informație perfectă – toate elementele sunt cunoscute ambilor jucători
- Secvențiale – jucătorii joacă pe rând

## Jocuri cu 2 adversari - Minimax

Jucătorul („eu”) caută să își maximizeze câștigul. Adversarul caută să minimizeze câștigul jucătorului. Calculăm pentru fiecare nod din arborele de joc recompensa bazat pe ideea că se poate alege cea mai bună mutare pentru orice stare a jocului, dacă avem informație perfectă.

Spațiul de căutare poate fi foarte mare și parcurgerea complet nepractică. Putem construi arborele doar până la o adâncime n. Mai departe de adâncimea n, evaluăm nodurile folosind o funcție de evaluare.

**Efectul de orizont** (horizont effect) – este posibil ca foarte curând după adâncimea n progresul jocului să se schimbe semnificativ.

**Minimax**

Intrări: $S$ – starea curentă a jocului, e.g. cea inițială;
$P$ – jucătorul care urmează, inițial MAX
$n$ – adâncimea maximă
Ieșire: scorul maxim care poate fi obținut de jucătorul MAX

1. **dacă** $S$ este stare finală, **atunci întoarce** $scor(S)$ *relativ la jucătorul MAX*
2. **dacă** $nivel(S) = n$ **atunci întoarce** $eval(S)$
3. **dacă** $P = \text{MAX}$ **atunci** $v = -\infty$ **altfel** $v = \infty$
4. **pentru fiecare** $S_j \in succ(S)$
5. **dacă** $P = \text{MAX}$
6. **atunci** $v \leftarrow max(v, minimax(S_j, next_player(P)))$
7. **altfel** $v \leftarrow min(v, minimax(S_j, next_player(P)))$
8. **întoarce** $v$

## Jocuri cu 2 adversari – Tăiere alfa-beta

Nu este neapărat nevoie să construim întreg arborele de joc. Putem elimina (prune) anumite ramuri pentru că nu au cum să ofere o soluție mai bună.

Considerăm α cea mai bună valoare găsită pentru jucătorul MAX și β cea mai bună valoare găsită pentru jucătorul MIN.

α-tăiere – nu construim un subarbore copil al unui nod MIN cu v < α

β-tăiere – nu construim un subarbore copil al unui nod MAX cu v > β

**Alfa-beta**

Intrări: $S$ – starea curentă; $P$ – următorul jucător;
$\alpha$, inițial $-\infty$; $\beta$, inițial $\infty$
Ieșire: scorul maxim care poate fi obținut de jucătorul MAX

1. **dacă** $S$ este stare finală, **atunci întoarce** $scor(S)$ *relativ la jucătorul MAX*
2. **pentru fiecare** $S_j \in succ(S)$
3. **dacă** $P = \text{MAX}$ **atunci**
4. $\alpha \leftarrow max(\alpha, alfa\text{-}beta(S_j, next_player(P), \alpha, \beta))$
5. **dacă** $\alpha > \beta$ **atunci întoarce** $\beta$ *β cutoff*
6. **altfel**
7. $\beta \leftarrow min(\beta, alfa\text{-}beta(S_j, next_player(P), \alpha, \beta))$
8. **dacă** $\alpha > \beta$ **atunci întoarce** $\alpha$ *α cutoff*
9. **dacă** $P = \text{MAX}$ **atunci întoarce** $\alpha$ **altfel** $\beta$

Putem ordona (euristic) succesorii în așa fel încât să construim întâi cei mai buni succesori.

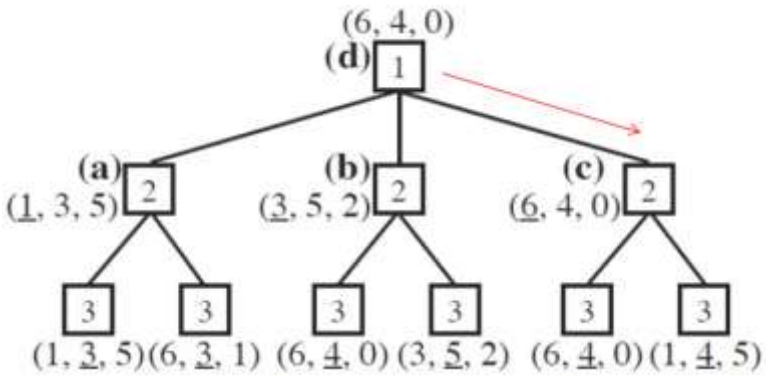
# Jocuri cu mai mulți jucători

Avem 2 strategii:

- MAX$^n$ – generalizare a Minimax pentru n jucători
- Paranoic – reduce la joc cu 2 jucători în care se presupune că toți ceilalți colaborează împotriva jucătorului MAX

# MAX$^n$

Generalizarea Minimax pentru n jucători. Nodurile arborelui de joc sunt n-tuple, în care elementul de pe elementul de pe poziția i este scorul jucătorului i. Pentru non-frunze, valoarea Max$^n$ a unui nod în care jucătorul i mută este valoarea Max$^n$ a succesorului pentru care a i-a componentă din vector este maximă.




Max$^n$
Intrări: $N$ – starea curentă;
$P$ – jucătorul care urmează să mute;
$n$ – adâncime maximă
Ieșire: scorul maxim care poate fi obținut de jucătorul MAX

1. **dacă** $N$ este stare finală **atunci întoarce** $N[P]$
2. **dacă** $nivel(N) = n$ **atunci întoarce** $eval(N, P)$
3. $next \leftarrow \{\}$
4. **pentru fiecare** $s_j \in succ(N)$
5. $\quad next \leftarrow next \cup Max^n(s_j, next_player(P))$
6. $best \leftarrow tuple \in next$, cu $tuple[P]$ maxim
7. **întoarce** $best$

Pot exista mai multe valori egale Max$^n$ într-un arbore. Rezultatul depinde drastic de felul în care se face alegerea. Alfa-beta în adâncime (deep pruning) nu poate fi aplicat.

**Problemă #21**. Care este principala diferență între strategia Max$^n$ și algoritmul Minimax tradițional pentru căutare în jocuri?

a. **Max$^n$ utilizează un vector de utilități în loc de o singură valoare de utilitate**
b. Max$^n$ nu necesită funcții de evaluare pentru stările non-terminale
c. Max$^n$ poate fi aplicat doar în jocuri cu sumă zero
d. Max$^n$ poate fi aplicat doar în jocuri cu informație incompletă

**Răspuns**. a. Max$^n$ utilizează un vector de utilități în loc de o singură valoare de utilitate

## Paranoic

Reduce jocul la 2 jucători, MAX și toți ceilalți jucători ca un adversar colectiv. Este o strategie paranoică. În fiecare nod scorul este scorul jucătorului MAX minus scorul tuturor ceilalți jucători. Fiind la fel ca Minimax:

- Există o unică valoare în fiecare nod
- Se poate utiliza tăiere alfa-beta, dar pe măsură ce numărul jucătorilor crește, beneficiul adus de tăiere scade
- Pentru jocuri cu 3-6 jucători, adâncimea este cu 20-50% mai mare decât la Max$^n$

**Problemă #22**. Avantajul strategiei Paranoic pentru un joc cu mai mult de 2 jucători este că:

a. **Consideră că toți ceilalți jucători se aliază împotriva jucătorului care are această strategie.**
b. În fiecare nod considerăm valorile individuale ale scorului pentru fiecare jucător
c. Produce arbori cu adâncime mai mică decât Minimax
d. Se poate utiliza tăierea alfa-beta, la fel ca la Minimax

**Răspuns**. a. Consideră că toți ceilalți jucători se aliază împotriva jucătorului care are această strategie. Strategia Paranoid (paranoică) este o variantă a algoritmului Minimax adaptată pentru jocuri cu mai mult de 2 jucători

**Problemă #23**. Din punct de vedere al eficienței computaționale. Care este avantajul principal al strategiei Paranoic față de strategia Max$^n$ într-un joc cu n jucători?

e. Strategia Paranoic nu necesită funcții de evaluare pentru nodurile non-terminale
f. Strategia Paranoic necesită evaluarea unui număr mai mic de stări în arborele de joc
g. **Strategia Paranoic reduce efectiv problema la un joc cu 2 jucători, permițând utilizarea algoritmului Minimax standard**
h. Strategia Paranoic garantează găsirea strategiei optime în timp liniar

**Răspuns**. c. Strategia Paranoic reduce efectiv problema la un joc cu 2 jucători, permițând utilizarea algoritmului Minimax standard.

# Monte Carlo Tree Search – MCTS

Algoritm probabilistic care utilizează o serie de simulări aleatoare pentru a expanda selectiv arborele de joc. Este o metodă bună pentru luarea deciziilor în probleme cu un spațiu de căutare mare. Metodele de tip Monte Carlo folosesc eșantionare aleatoare repetată pentru a obține informații despre procesele deterministe.

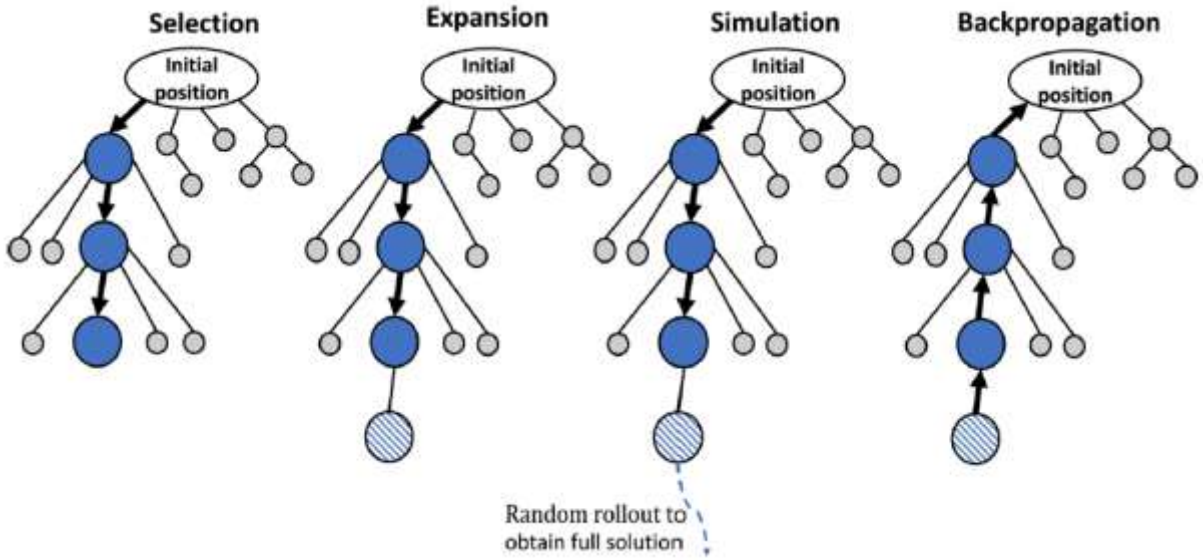
Se bazează pe 2 ipoteze:

- Adevărata valoare a unei acțiuni (mutare în joc) poate fi aproximată utilizând simulări aleatoare
- Valorile astfel obținute pot fi utilizare pentru a ajusta politica de selecție spre o cea mai bună strategie

- baza metodei este utilizarea unei unde de joc / simulări (*playout*).
- Playout – un joc jucat cu mutări aleatoare dintr-o anumită stare *până la starea finală*, obținându-se un scor.
  - nu utilizează spațiu suplimentar pentru că nu construiește noduri în arbore – în urma simulării se reține doar scorul final.
- fiecărui nod construit i se asociază un merit (sau calitate – *quality*), bazat pe rezultatul simulărilor.
- se construiește un arbore de joc parțial, format din stările cele mai promițătoare.
- algoritm de tip *stop-anytime* – ne putem opri oricând și avem o soluție, care este cu atât mai bună cu cât am explorat mai mult.

La fiecare iterație a algoritmului avem 4 pași:

- Selecție – căutăm un nod care nu este încă (complet) expandat – pornim de la rădăcină și aplicăm recursiv o politică de selecție a copiilor pentru a găsi cel mai potrivit nod căruia să îi construim un copil
- Construcție (expandare) – construim unul sau mai multe noduri, corespunzătoare unor acțiuni încă neexplorate
- Simulare – realizăm o simulare pornind din nodul / nodurile noi
- Propagare – propagăm către rădăcină rezultatul obținut




MCTS(rădăcină)

1. $(v, s) \leftarrow$ nodul rădăcină, corespunzător stării $s_0$, $parinte(v) \leftarrow \perp$
2. **pentru fiecare** unitate de buget **repetă**
3. **cât timp** $s$ nu este stare finală
4. **dacă** $v$ este complet expandat **atunci** $(v, s) \leftarrow select(v)$ **altfel break**
5. $a \leftarrow alege_actiune(s)$ *am ajuns la nodul care trebuie expandat*
6. construiește $(v', s')$, cu $s' \leftarrow next(s, a)$ și $parinte(v') \leftarrow v$; $(v, s) \leftarrow (v', s')$
7. **cât timp** $s$ nu este stare finală
8. $a \leftarrow$ acțiune disponibilă în $s$, aleasă aleator
9. $s \leftarrow next(s, a)$
10. $r \leftarrow recompensa(s')$
11. **cât timp** $v \neq \perp$
12. $N(v) \leftarrow N(v) + 1$
13. $Q(v) \leftarrow actualizare_Q(v, r)$
14. $v \leftarrow parinte(v)$
15. $(v, s) \leftarrow select(radacina)$
16. **întoarce** $a$, cu $s = next(s_0, a)$ *cea mai bună acțiune*

Ce strategii se folosesc pentru selecție?

- Exploatare – selectăm stări din care s-a câștigat des și au fost parcurse de mai multe ori
- Explorare – selectăm stări cu puține simulări anterioare

- Selecție folosind UCT – *Upper Confidence Bound 1 applied to trees*

$$select(s) \leftarrow argmax_{s' \in succ(s)} \left( \frac{Q(s')}{N(s')} + C \cdot \sqrt{\frac{2 \cdot \ln(N(s))}{N(s')}} \right)$$

- Echilibrează calitatea unui nod copil cu gradul de explorare al părintelui în raport cu copilul.

- Backpropagation poate folosi, pentru propagarea de la $s'$ la părintele $s$:

$$V(s) \leftarrow \frac{V_{med} \cdot W_{med} + V_r \cdot N_r}{W_{med} \cdot N_r}$$

  - $V_{med}$ — media valorilor nodurilor copii
  - $W_{med}$ — ponderea acestei medii
  - $V_r$ — mutarea cu cel mai mare număr de simulări
  - $N_r$ — numărul de ori de care s-a jucat $V_r$

- MCTS converge *lent* spre valorile Minimax, dar este mai eficient decât tăierea alfa-beta.

- nu găseste întotdeauna cea mai bună mutare, dar are, în general, un succes rezonabil în cazul alegerii mutărilor care duc la șanse mari de câștig.

- poate să nu detecteze ramuri care, folosind *o anumită line de joc*, duc la un rezultat semnificativ diferit

- *Light playouts* — playout complet aleator

- *Heavy playouts* — euristici/politici pentru selectarea miscării următoare în timpul simulării

**Problemă #24**. Care dintre următoarele afirmații este adevărată despre algoritmul MCTS?

a. Selecția unui nod pentru expandare dă prioritate explorării în defavoarea exploatării
b. Simularea dă prioritate exploatării în alegerea stărilor
**c. Selecția acțiunii de realizat din rădăcină dă prioritate exploatării**
d. Explorarea și exploatarea sunt perfect balansate în orice decizie a algoritmului

**Răspuns**. c. Selecția acțiunii de realizat din rădăcină dă prioritate exploatării. După multe simulări, se alege mutarea cu cel mai bun rezultat mediu (prioritizează exploatarea).

**Problemă #25**. Care dintre următoarele afirmații este adevărată despre algoritmul MCTS?

**a. În pasul de expandare nodurile copii se vor genera doar pentru acțiuni neexplorate din acea stare**
b. Nu poate fi aplicat în probleme cu un spațiu de căutare mare
c. Etapa de propagare înapoi actualizează toate nodurile din arbore
d. Pentru a putea genera o mutare, arborele MCTS trebuie să aibă primul nivel complet explorat

**Răspuns**. a. În pasul de expandare nodurile copii se vor genera doar pentru acțiuni neexplorate din acea stare.

**Problemă #26**. În implementarea algoritmului MCTS pentru un joc adversarial cu factor de ramificare mare, cum ar fi Go, care dintre următoarele reprezintă cel mai important avantaj față de algoritmul Minimax cu tăieri alfa-beta?

a. MCTS necesită cunoștințe de expert pentru a defini funcția de evaluare statică
b. MCTS are o complexitate spațială mai redusă dar niciodată o performanță practică mai bună
c. MCTS garantează găsirea strategiei optime într-un timp finit pentru orice joc adversarial
**d. MCTS poate oferi decizii de calitate rezonabilă cu simulări limitate**

**Răspuns**. d. MCTS poate oferi decizii de calitate rezonabilă cu simulări limitate. Principalul avantaj al MCTS în jocuri ca Go este că poate lua decizii bune chiar și cu resurse limitate, fără a necesita o funcție de evaluare perfectă sau explorarea completă a arborelui.

**Problemă #27**. Care dintre următoarele afirmații este adevărată despre algoritmul MCTS?

a. Propagarea înapoi trece prin toate nodurile arborelui generat
b. Selecția unui nod pentru expandare se face aleator
c. MCTS explorează toate stările pentru a genera acțiunea următoare
**d. Simularea unui joc pentru evaluarea unei stări se face aleator**

**Răspuns**. d. Simularea unui joc pentru evaluarea unei stări se face aleator. Simularea (playout/rollout) se face tipic prin mutări aleatoare sau folosind o politică simplă.

**Problemă #28**. În algoritmul MCTS, formula UCB (Upper Confidence Bound) este utilizată în faza de selecție. Care este rolul principal al termenului de explorare din formula UCB?

a. Favorizează nodurile care au fost vizitate mai frecvent
**b. Echilibrează compromisul între explorare și exploatare**
c. Reduce timpul total de execuție al algoritmului
d. Generează convergența la strategia optimă în timp finit

**Răspuns**. b. Echilibrează compromisul între explorare și exploatare.